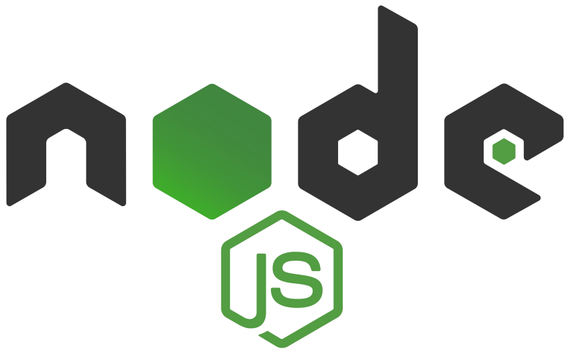
iteam | INSTITUTO TECNOLÓGICO EDUCACIONAL DA AMAZÔNIA

Paixão por Desenvolver Talentos

Introdução ao Node.js

Módulo 05: Desenvolvimento Back-end com Node.js e Express
Instrutor: Janei Vieira
Aula 1: Setup do Ambiente



Objetivos da Aula

- Compreender a história e evolução do Node.js
- Entender o funcionamento do **motor V8** e **Event Loop**
- Configurar o ambiente de desenvolvimento
- Criar a primeira aplicação prática em Node.js

iteam | INSTITUTO TECNOLÓGICO EDUCACIONAL DA AMAZÔNIA
Paixão por Desenvolver Talentos

iteam | INSTITUTO TECNOLÓGICO EDUCACIONAL DA AMAZÔNIA

Paixão por Desenvolver Talentos

Janei Vieira

 Full Stack Developer 👨‍💻

🎉 24 anos de experiência em tecnologia. Especialista em infraestrutura, automação, containers (Docker/Kubernetes) e desenvolvimento back-end para sistemas comerciais e aplicações em .NET C#, Python e Node.JS

👨 Foco em Front-ends com Razor Pages e Streamlit e Flask



Histórico Acadêmico

- **Graduação:** Gestão da Tecnologia da Informação.
- **MBA:** Em Gestão Ágil de Projetos e Processos
- **Especializações:**
  - Redes e Computação Distribuída (**IFMT**).
  - Engenharia DEVOPS (**IFMT**)
  - Big Data (Ciência de Dados).
  - Computação Em Nuvem (Cloud Computing).

Mestrando: Propriedade Intelectual para Transferência de Tecnologia e inovação (**UFRR**)

https://lattes.cnpq.br/4083863936027227

iteam | INSTITUTO TECNOLÓGICO EDUCACIONAL DA AMAZÔNIA
Paixão por Desenvolver Talentos

Antes de Programar

Pré-Requisitos antes do Node.JS

- **LÓGICA:** ESSENCIAL -> Como estruturar raciocínio lógico em forma de app
- **JAVASCRIPT:** BOM SABER -> A linguagem core do Node.js
- **TYPESCRIPT:** OPCIONAL -> Um Superset que adiciona novas possibilidades ao JS

Contexto Histórico

O Problema C10k

- **Cenário pré-2009:** Servidores tradicionais (como o Apache) criavam uma nova thread (linha de execução) para cada conexão de usuário.
- **O Problema:** Isso consumia muita memória RAM e CPU, limitando a escalabilidade. Era o chamado "C10k Problem" (dificuldade em lidar com 10 mil conexões simultâneas).
- **A Necessidade:** Era preciso uma forma de lidar com milhares de conexões de forma leve e eficiente.

Contexto Histórico

O Nascimento do Node.js

- **Criador:** Ryan Dahl, em **2009**.
- **A Inspiração:** Dahl percebeu que o I/O (Entrada/Saída) era o maior gargalo e que o JavaScript, por ser orientado a eventos, era a ferramenta perfeita para resolver isso.
- **O Motor:** Ele pegou o V8 Engine (o motor de JavaScript super-rápido do Google Chrome) e o trouxe para o lado do servidor.

Contexto Histórico

O Nascimento do Node.js

2010 - A versão 0.1.14 do Node.js foi lançada, marcando o início do desenvolvimento da comunidade e o crescente



Contexto Histórico

O Nascimento do Node.js

2011 – O Node.js foi adotado por grandes empresas como Microsoft, Yahoo! E LinkedIn, aumentando ainda mais sua popularidade.



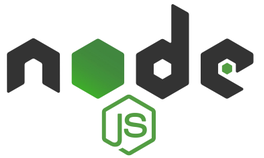
Contexto Histórico

O Nascimento do Node.js

2012 - A formação da Fundação Node.js como uma organização sem fins lucrativos para gerenciar e evoluir o projeto. Mais tarde conhecida como OpenJS Foundation



OpenJS Foundation



Contexto Histórico

O Nascimento do Node.js

2013 - Lançamento da versão 0.10 do Node.js, trazendo melhorias de desempenho e estabilidade



Contexto Histórico

O Nascimento do Node.js

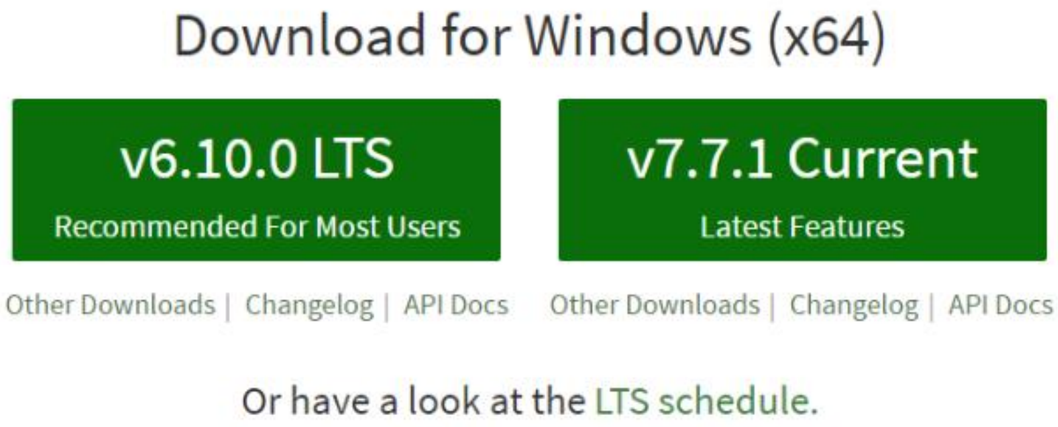
2015 - A versão 0.12 introduziu o suporte oficial para o ECMAScript 6 (ES6), trazendo recursos modernos ao JavaScript



Contexto Histórico

O Nascimento do Node.js

2016 - O lançamento da versão 6 do Node.js, marcando a transição para um ciclo de lançamento de versões Long Term Support (LTS)



Contexto Histórico

O Nascimento do Node.js

2020 - Ryan Dahl cria o Deno, o maior concorrente do Node.js, dessa vez fazendo tudo do jeito como ele queria.



O que o Node não é.

Não é uma Linguagem

Node.js não é uma linguagem de programação própria. A linguagem utilizada é o **JavaScript**, executada fora do navegador.

Não é um Framework

Diferente de Express ou NestJS, o Node.js é um **Runtime** (ambiente de execução) que permite ao JS rodar no servidor.

Não é Limitado

O Node.js não é restrito a tarefas simples. Ele é capaz de realizar tudo o que outras **tecnologias** de back-end (Java, PHP, Python) fazem.

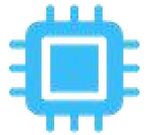
O que dá pra fazer com Node?



REAL-TIME CHATS
Aplicações de mensagens instantâneas e colaboração em tempo real com WebSockets.



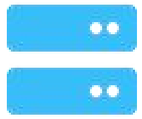
DATA STREAMING
Processamento de grandes volumes de dados em fluxo, como serviços de vídeo e áudio.



IOT APPLICATIONS
Integração com dispositivos inteligentes e sensores devido ao baixo consumo de recursos.



WEB APPLICATIONS
Servidores web escaláveis e rápidos para lidar com milhares de conexões simultâneas.



REST API SERVER
Construção de APIs robustas e performáticas para servir dados a diferentes clientes.



COMPLEX SPAS
Back-end ideal para Single Page Applications modernas construídas com React ou Vue.

iteam | INSTITUTO TECNOLÓGICO EDUCACIONAL DA AMAZÔNIA
Paixão por Desenvolver Talentos

Como Funciona


= Rendering Engine + JavaScript Engine + User Interface

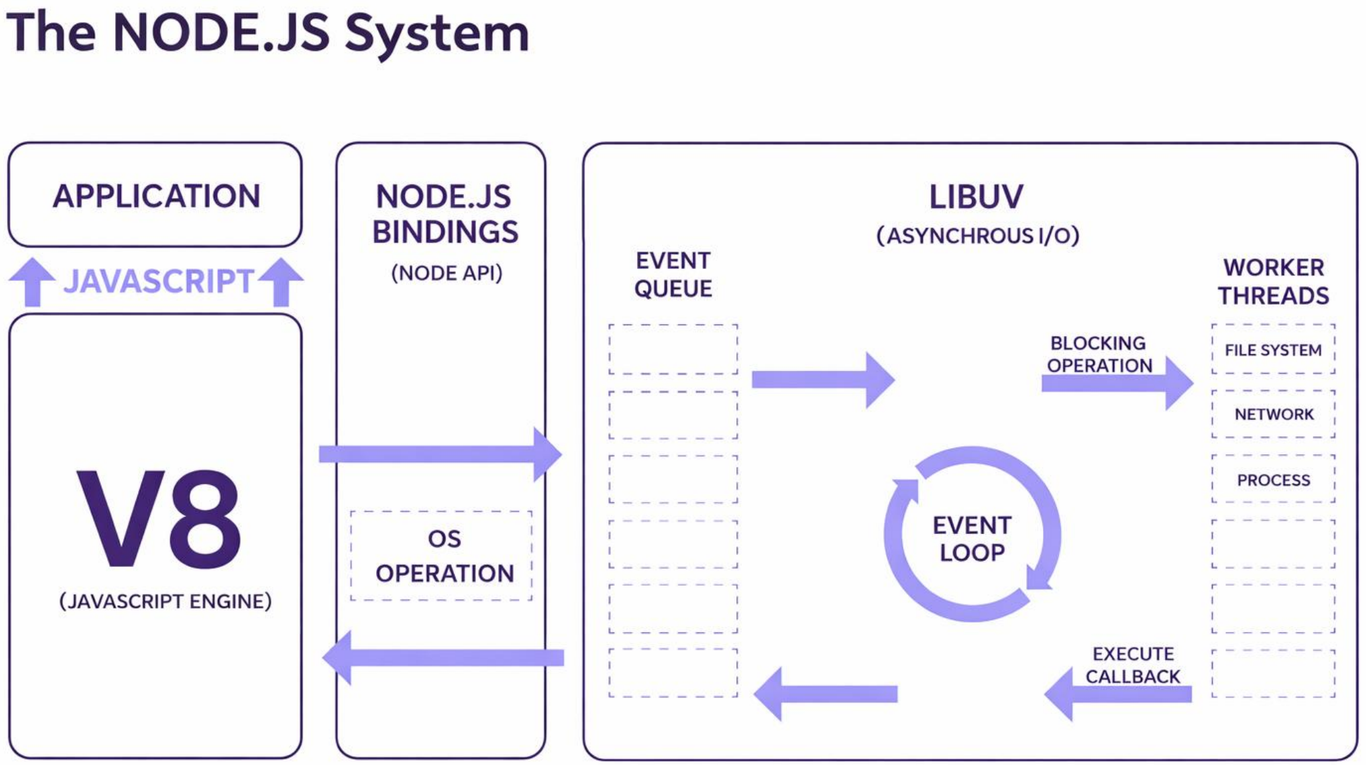

= Trident + Chakra


= Gecko + *Monkey


= Webkit + V8

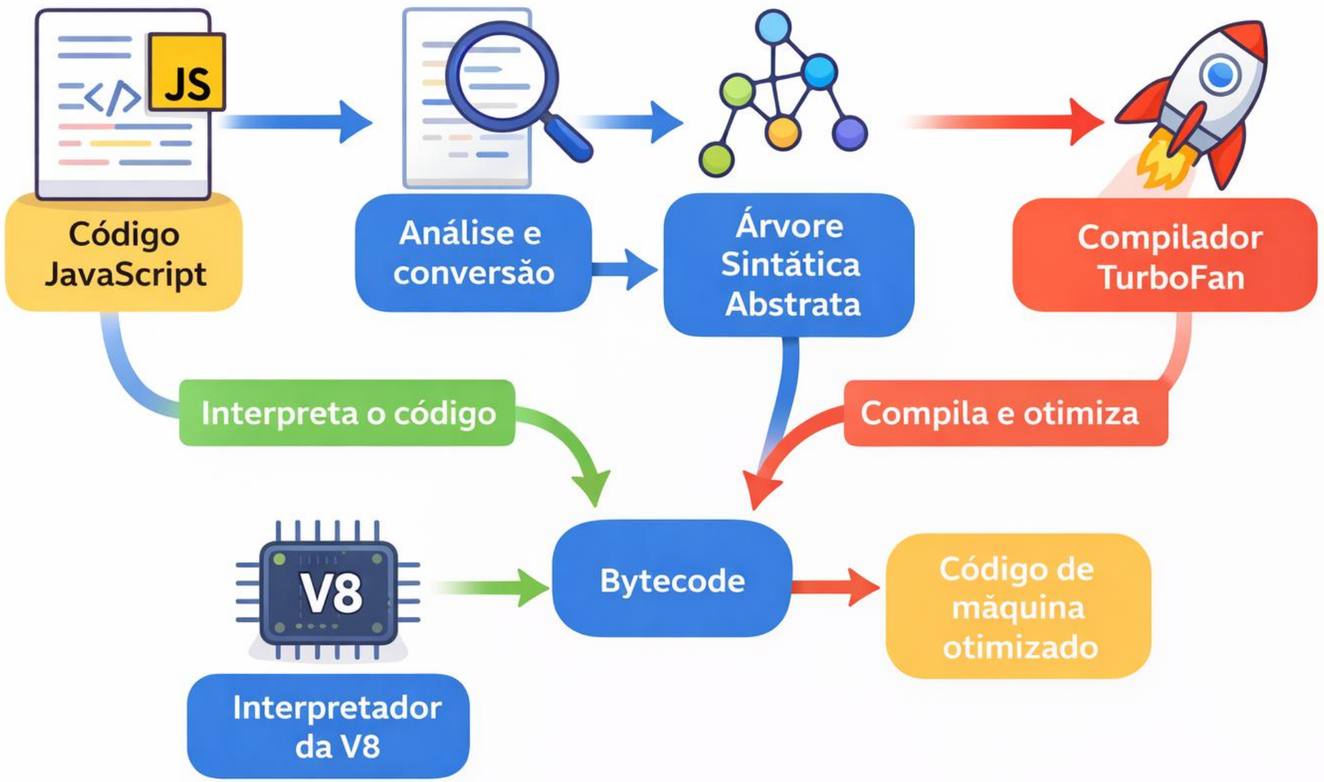

= Webkit + JavascriptCore

Como Funciona



How NODE.JS works: JavaScript runs on the V8 engine, uses Node.js Bindings to handle OS operations, events go into an event loop, and LIBUV's worker threads manage blocking tasks like file, network, and process operations.

Simplificando



V8 ENGINE

O Coração do Node.Js



O que é: Desenvolvido pelo **Google** para o Chrome, o V8 é um motor de JavaScript e WebAssembly de código aberto, escrito em **C++**. Ele é o responsável por dar ao Node.js sua velocidade lendária.

Como Funciona: O V8 compila o JavaScript diretamente para **código de máquina nativo** antes de executá-lo, em vez de interpretá-lo em tempo real. Esse processo, conhecido como **JIT (Just-In-Time)**, elimina a sobrecarga da interpretação.



Arquitetura

Event Loop e I/O Não Bloqueante

- **Non-blocking I/O:** O Node.js não espera uma operação de banco de dados ou leitura de arquivo terminar para atender o próximo usuário; ele delega e continua trabalhando.
- **Event Loop:** Um ciclo que monitora e executa tarefas de forma assíncrona, garantindo alta disponibilidade mesmo com poucos recursos de hardware.
- **Single-threaded:** Diferente do modelo antigo, o Node usa uma única thread principal para gerenciar todas as requisições de forma inteligente

Vantagens e Casos de Uso

- **Escalabilidade:** Ideal para aplicações de tempo real (chats, jogos, streaming)
- **Ecossistema:** Acesso ao NPM (Node Package Manager), o maior repositório de bibliotecas do mundo.
- **Leveza:** Menor consumo de recursos em comparação com tecnologias como Java ou .NET para APIs REST.

Preparando o Ambiente

- **Download:** Instalação oficial via **nodejs.org.**
- **Versão LTS (Long Term Support):** Focada em estabilidade e segurança para produção.
- **Verificação:** Comandos essenciais no terminal para validar a instalação:
  - node -v (verifica a versão do runtime).
  - npm -v (verifica o gerenciador de pacotes).

Prática

Nosso Primeiro Script

- **Passo 1:** Criar uma pasta para o projeto e abrir no editor (ex: VS Code).
- **Passo 2:** Criar um arquivo chamado **app.js**.
- **Passo 3:** Escrever o código inicial: **console.log("**Servidor Node.js iniciado com sucesso!**");**
- **Passo 4:** Executar via terminal com o comando **node app.js**

iteam | INSTITUTO TECNOLÓGICO EDUCACIONAL DA AMAZÔNIA

Paixão por Desenvolver Talentos

Até a Próxima Aula!

iteam | INSTITUTO TECNOLÓGICO EDUCACIONAL DA AMAZÔNIA

Paixão por Desenvolver Talentos

node JS